SDLC Using Agentic AI

Building an Adaptive AI Software Engineering Organization

Presented By:

Abhishek Raj (M.Tech Intern)

Guide:

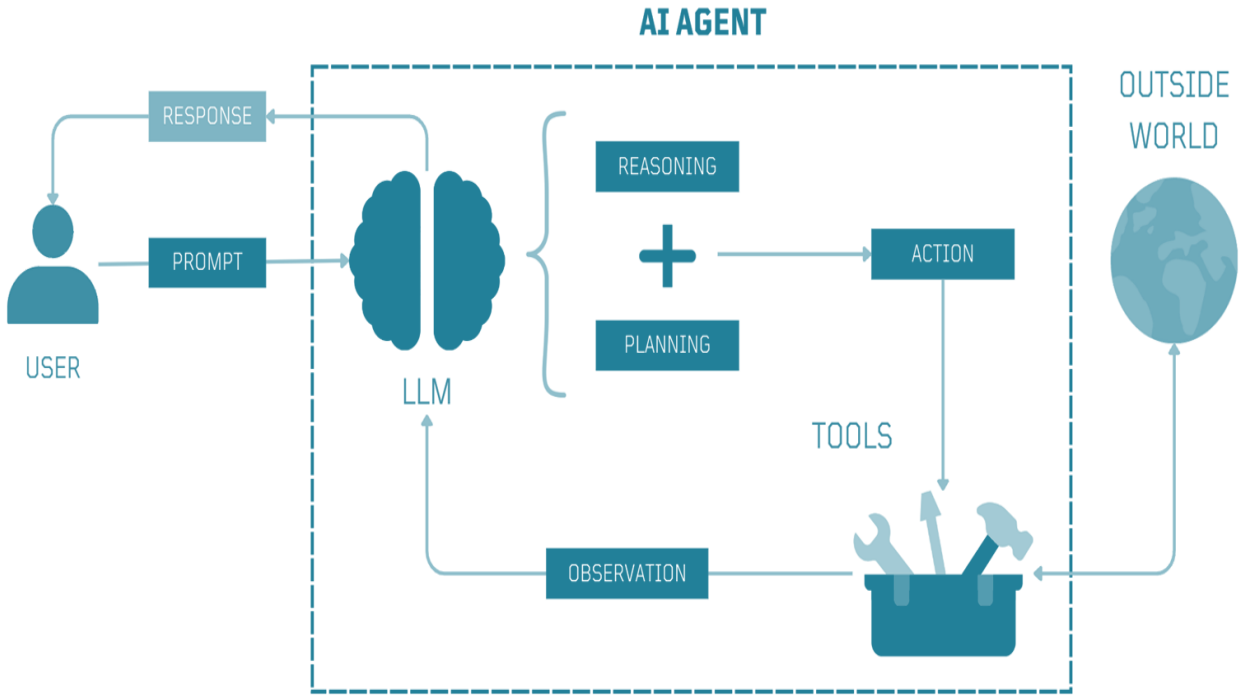
M.B. Sai Charan (Scientist C)

What is an AI Agent?

An AI agent is a system where an LLM operates in a loop: it receives observations from an environment (user messages, tool outputs, sensor data), reasons about what to do next, takes actions (tool calls, code execution, API requests), and iterates until a goal is achieved or it explicitly asks for human input.

An AI agent should have the following characteristics:

- **Persistence:** An agent must remember what it has done, what failed, and what context was established—across turns, sessions, and even days.
- **Grounding:** The agent must access up-to-date, domain-specific knowledge that was not present in its training data.
- **Action:** The agent must interact with external systems—databases, APIs, file systems, browsers—through well-defined interfaces.
- **Coordination:** Complex tasks often exceed what a single agent can handle; multiple specialized agents must collaborate, delegate, and negotiate.
- **Safety:** Autonomous action requires guardrails, human oversight, and graceful degradation when the agent is uncertain. [6]



Architectures

Workflow vs Autonomous Agent Design Patterns

Workflow Patterns

Execution paths, step ordering, and condition gates are explicitly defined in code. LLM calls serve as reliable processing nodes within the system pipeline. [6]

- Higher predictability and testability
- Lower latency and controlled token expenditure
- Ideal for standardized, step-by-step transformations

> **Primary Patterns:** Prompt Chaining, Sequential Workflows, Routing, Parallelization, Orchestrator-Workers, Evaluator-Optimizer.

Autonomous Agent Pattern

The agent autonomously determines its execution path based on environment feedback, goal planning, reflection, and tool execution. [6]

- High adaptability to open-ended goals
- Dynamic tool selection and runtime recovery
- Handles complex multi-step reasoning environments

> **Primary Patterns:** ReAct (Reason+Act), Planning Agents, Reflection / Self-Critique, Multi-Agent Collaboration.

Workflow Patterns

1. Prompt Chaining

Decomposes a task into a fixed sequence of LLM calls where the output of one step becomes the input context for the next, with validation gates, ensuring higher precision and intermediate quality control at each stage. [6]

When to use: Tasks that are naturally sequential—content generation, data transformation, multi-stage analysis.

> **Use Cases:** Multi-stage reasoning, document analysis, data transformation.

Fig: Prompt chaining with quality gates. Each step is a separate LLM call. Gates can be LLM-based or programmatic.[6]

2. Routing

Uses an initial router or classifier to direct incoming requests to specialized agents or processing paths with tailored prompts and tools , optimizing latency and cost by serving simple requests quickly and reserving larger, specialized models for complex tasks. [6]

When to use: Distinct task types with different optimal prompts, tools, or models. Customer support triage, multi-modal input handling.

> **Use Cases:** Customer support triage, query classification

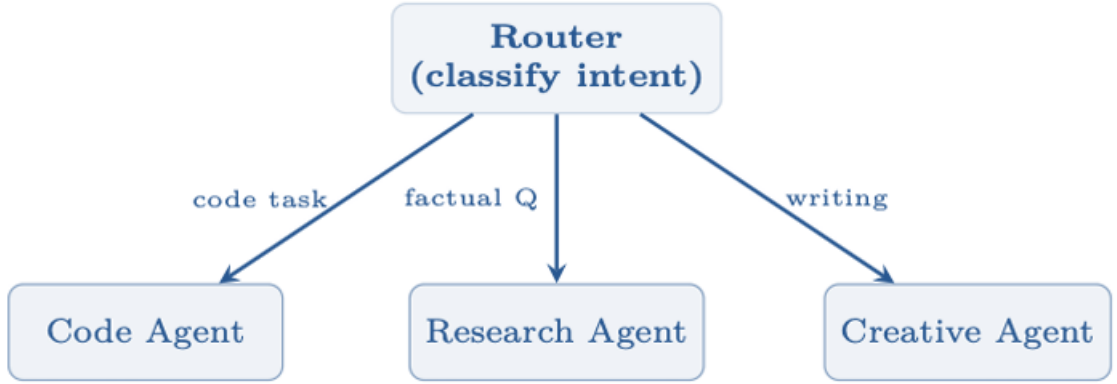


Fig: Routing pattern: input is classified once, then handled by a specialist. [6]

Workflow Patterns (Cont.)

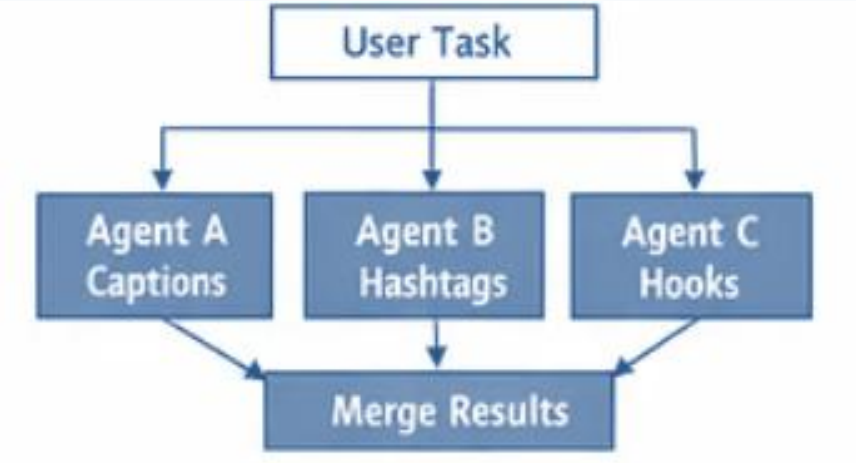
4. Parallelization

Distributes independent subtasks across concurrent LLM calls. Executes simultaneously and aggregates outputs to reduce total system latency , making it ideal for independent evaluation tasks, majority-voting consensus mechanisms, and multi-perspective verification. [6]

It has two sub-patterns:

Sectioning (fan-out): Partition the input into disjoint chunks and process each independently— e.g., run security, performance, and style checks on a codebase simultaneously.

Voting (redundancy): Issue the same prompt N times with different seeds or temperatures, then select the best result via majority vote [368], reward-model scoring, or LLM-as-judge.



> **Use Cases:** Multi-aspect code reviews, guardrail checks.

Workflow Patterns (Cont.)

5. Orchestrator–Workers

A central LLM dynamically analyzes a task, breaks it down into subtasks, delegates to specialized worker agents, and synthesizes the collected worker outputs into a final result. [6]

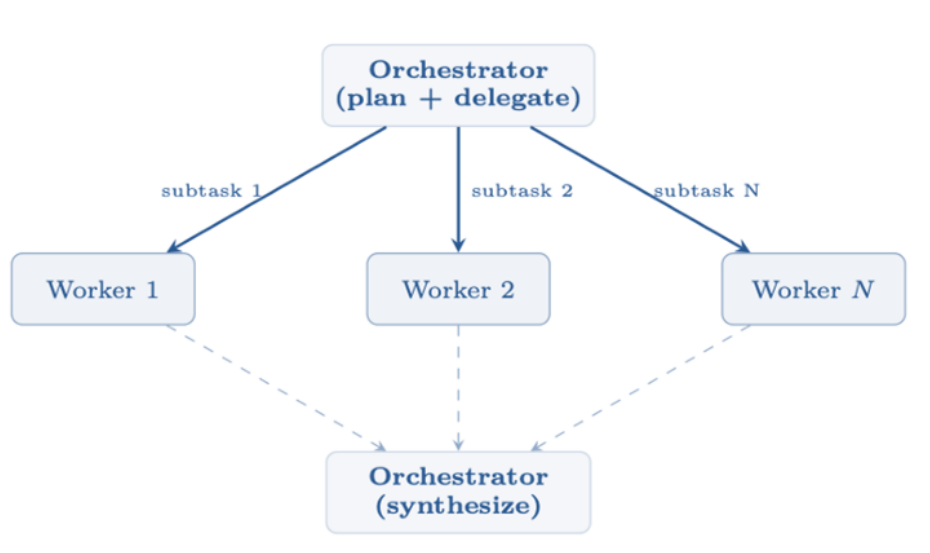


Fig: Orchestrator-workers: the LLM decides how to decompose the task and synthesizes worker results. [6]

> **Example Use Cases:** Complex software development, multi-domain compliance analysis.

6. Evaluator–Optimizer

Creates an iterative loop where a Generator LLM produces output and an Evaluator LLM assesses it against quality criteria, providing actionable feedback until thresholds are met.[6]

When to use: Tasks with clear quality criteria—code that must pass tests, translations that must preserve meaning, writing that must match a style guide.

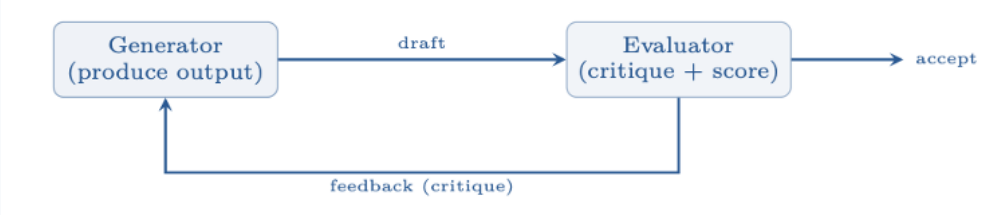


Fig: Evaluator-optimizer: iterative refinement without training.[6]

> **Example Use Cases:** Code generation with test pass verification, high-stakes translation,

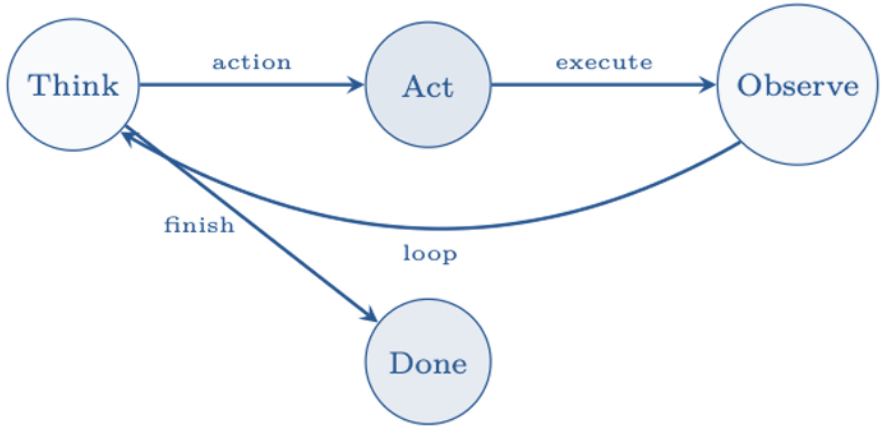
Autonomous Agent Patterns

Systems where the LLM-Based Agent independently determines execution paths, tool selection, reflection loops, and agent collaboration.

1. ReAct (Reason & Act)

It combines reasoning and action in a dynamic, iterative loop where a single agent observes the environment, formulates a plan, executes an action using available tools or APIs, and evaluates the resulting feedback to decide the next step. By constantly re-evaluating its context, the model can course-correct on the fly, retry failed tool calls, or adjust its strategy based on intermediate outputs. [6]

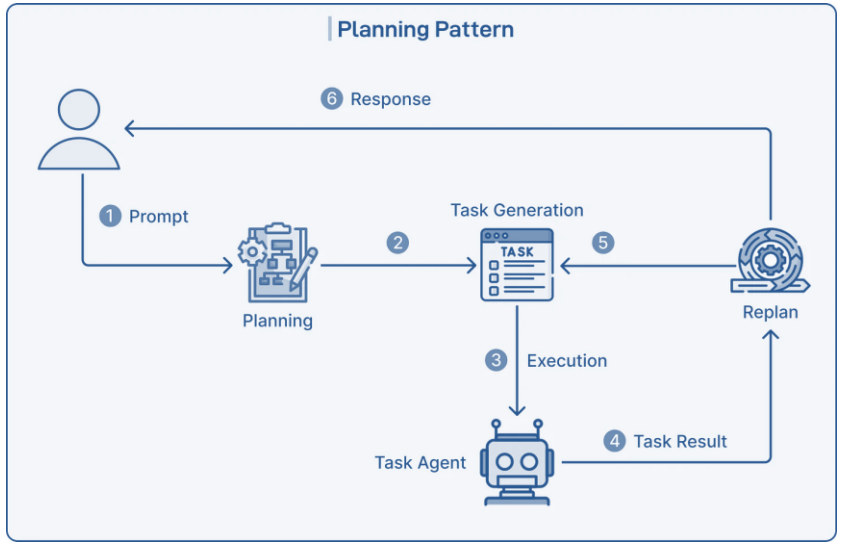
$$\text{Thought}_t \rightarrow \text{Action}_t \rightarrow \text{Observation}_t \rightarrow \text{Thought}_{t+1} \rightarrow \cdots$$



Example Use Cases: Q&A, Code generation & debugging

2. Planning Agent

It generates an explicit plan for achieving a goal before beginning execution. The plan contains tasks and dependencies, while the agent can revise the plan when new information, failures, or unexpected results are encountered during execution.[6]



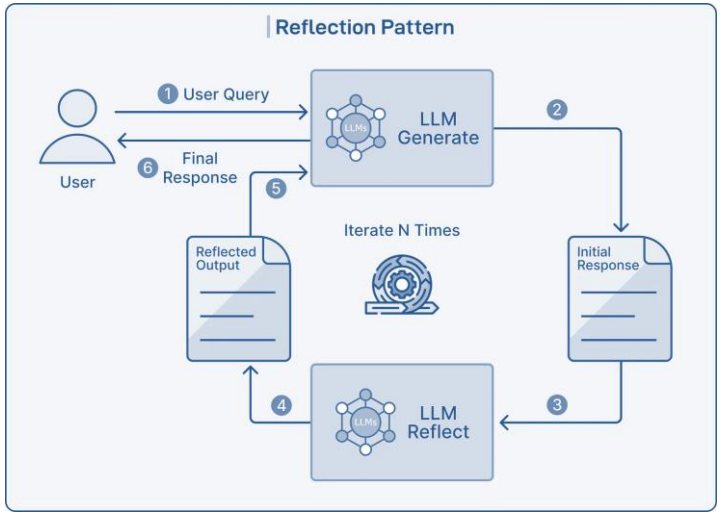
Example Use Cases: Content generation, document analysis, data transformation

Autonomous Agent Patterns (Cont.)

Systems where the LLM-Based Agent independently determines execution paths, tool selection, reflection loops, and agent collaboration.

3. Reflection & Self-Critique

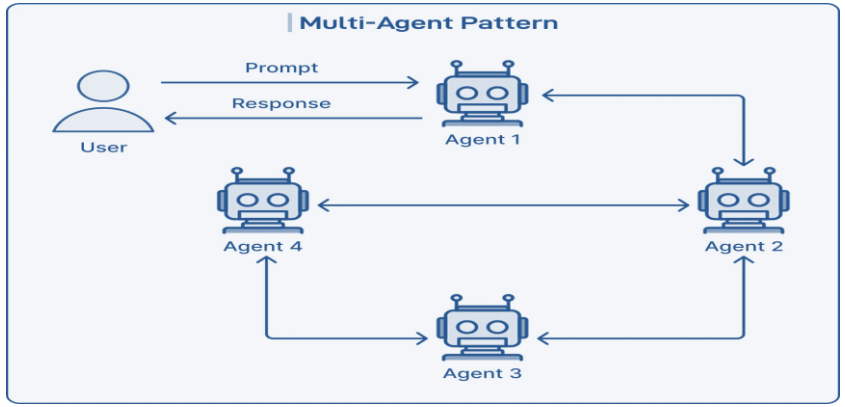
It introduces a self-evaluation stage where the agent examines its previous actions or output, identifies errors or inefficiencies, and modifies its strategy. The reflection can also be stored as memory and reused during subsequent attempts [6]



Example Use Cases: Customer support automation

4. Multi-Agent Swarm

A **Multi-Agent Swarm** relies on a manager agent to orchestrate a ecosystem of domain-expert sub-agents working together toward a high-level goal. The manager breaks down complex user objectives into specific tasks and assigns them to specialized roles—such as a Coder for implementation, a Researcher for context gathering, and a Reviewer for quality assurance—who can debate, review each other's work, and iteratively refine deliverables. [6]



Example Use Cases: Software development teams, Threat analysis & support

Agent Harness

LLM-based agents do not operate in isolation. Between the raw language model and the real-world tasks it must accomplish lies a layer of infrastructure that manages memory, routes tool calls, tracks state, and enforces safety constraints. This infrastructure is called the **agent harness.**

An **agent harness** is the runtime infrastructure that wraps an LLM to transform it from a stateless text-completion engine into a stateful, goal-directed agent capable of multi-step reasoning, tool use, memory retrieval, and interaction with external systems.[6]

The harness enforces a clean separation of concerns:

- **Reasoning:** Delegated entirely to the LLM; the harness does not second-guess model outputs.
- **Execution:** The harness dispatches tool calls, manages I/O, and enforces sandboxing.
- **Memory:** The harness maintains short-term (context window), working (scratchpad), and long-term (vector store / database) memory.
- **Communication:** The harness handles message routing between agents, users, and external services.
- **Observability:** The harness instruments every step for logging, tracing, and debugging [6]

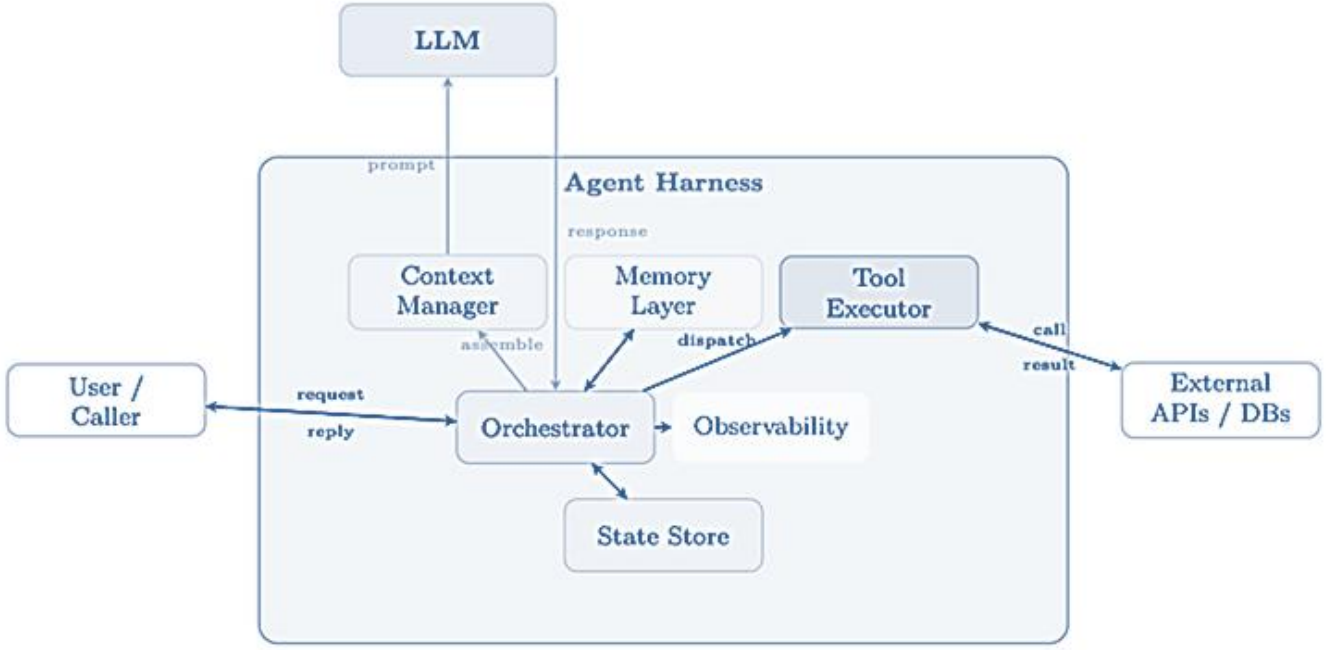
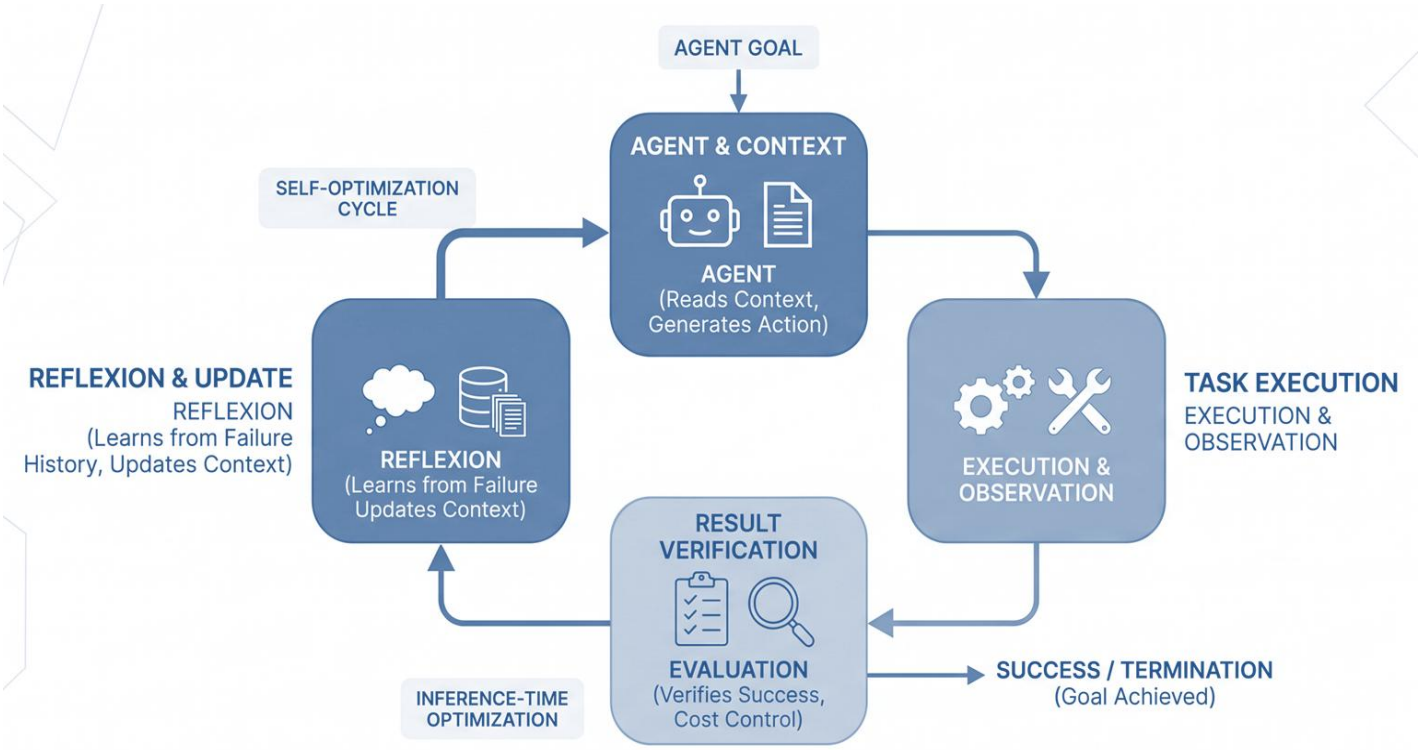


Fig: High-level architecture of an agent harness. The LLM handles only reasoning; all execution, memory, routing, and observability are managed by the harness. [6]

Loop Engineering

The **Orchestration Harness** coordinates everything: it decides when to retrieve, when to call tools, when to delegate to sub-agents, and when to ask the human for guidance, whereas **Loop Engineering** formalizes the harness's retry-and-verify cycles as inference-time optimization, ensuring convergence and cost control.

It reflects a structural reality: When a single agent run may last an hour and modify dozens of files, the highest-leverage engineering is no longer in the prompt—it is the **Loop** that keeps the agent productive, verified, and on-goal throughout. [6]



From Prompt to Loop: The Engineering Progression[6]

1. **Prompt engineering:** Optimize what you say to the model (2022–24)
2. **Context engineering:** Optimize everything the model sees (2025)
3. **Harness engineering:** Optimize the environment the agent runs in (2025–26)
4. **Loop engineering:** Optimize the cycle that keeps the agent working toward a goal (2026)

Loop Engineering

A well-engineered agent loop is structurally identical to an RL optimization process operating at inference time—without gradient updates to the model weights:

$$
\begin{aligned}
s_t &= \text{context}(s_{t-1}, a_{t-1}, o_{t-1}) && \text{(state = accumulated context)} \\
a_t &= \pi_\theta(s_t) && \text{(action = model generation)} \\
o_t &= \text{env}(a_t) && \text{(observation = tool/environment feedback)} \\
r_t &= \text{verify}(s_t, a_t, o_t) && \text{(reward = verification signal)}
\end{aligned}
$$

The **loop** continues until $\mathbf{r_t}$ exceeds a success threshold or a termination condition triggers. The "policy" $\boldsymbol{\pi_\theta}$ is not updated via gradients; instead, the ***state*** is updated—observations, error messages, and reflections are appended to the context, conditioning the same frozen model to produce better actions on subsequent iterations. This is precisely the mechanism behind **Reflexion**, the model improves across iterations not by learning new weights but by reading its own failure history. [6]

Reflection in Practice (Reflexion)

After three failed attempts to solve a coding problem, the agent reflects:

1. Retrieves the three failure episodes from episodic memory.
2. Generates an insight: "I keep forgetting to handle the edge case where the input list is empty."
3. Stores this insight in semantic memory.
4. On the next attempt, retrieves the insight and explicitly checks for empty inputs.

This is the core mechanism of **Reflexion**; learning via self-reflection. [6]

MCP & Why Do We Need It?

- **The Core Problem:** Every time a new LLM agent framework appears, developers must re-implement connectors to the same tools: file systems, databases, web search, code execution, calendar APIs. This is wasteful, error-prone, and creates a maintenance burden that scales quadratically with the number of agents and tools.

  Also LLM knowledge is frozen at training time & LLMs Agents cannot inherently interact with the outside world (real-time data, business systems, APIs).

- **The Solution (MCP):** The Model Context Protocol is an open standard that enables developers to build secure, two-way connections between their data sources and AI-powered tools. The architecture is straightforward: developers can either expose their data through MCP servers or build AI applications (MCP clients) that connect to these servers [18]

Here is a simplified look at how MCP would handle this:

1. Discover Tools: AI realizes it needs external help and finds the database and email tools registered on MCP.

2. Fetch Data: AI requests the sales report via MCP; the database server fetches and returns the report.

3. Send Email: AI hands the report to the email tool, which sends it to the manager.

4. Confirm: AI tells you the task is finished. [18][6]

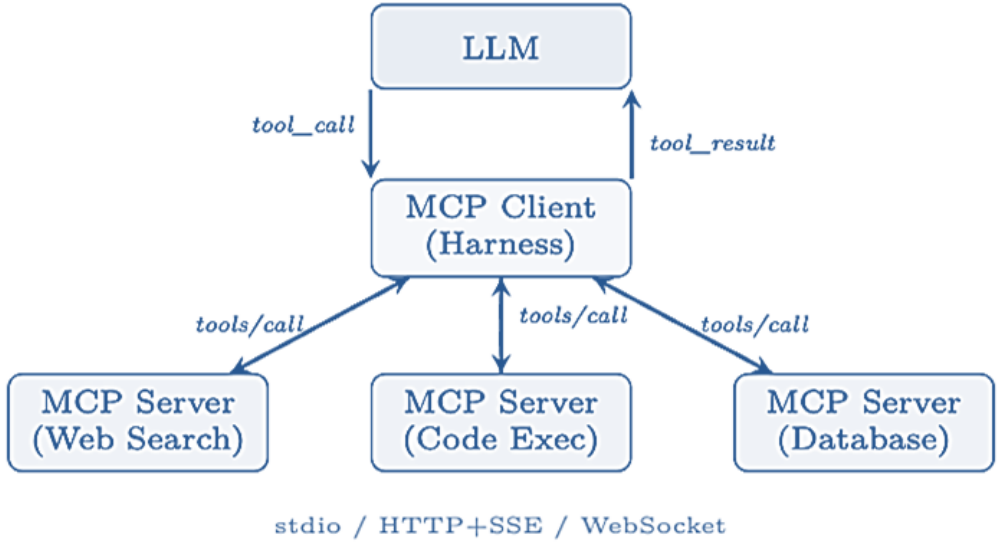
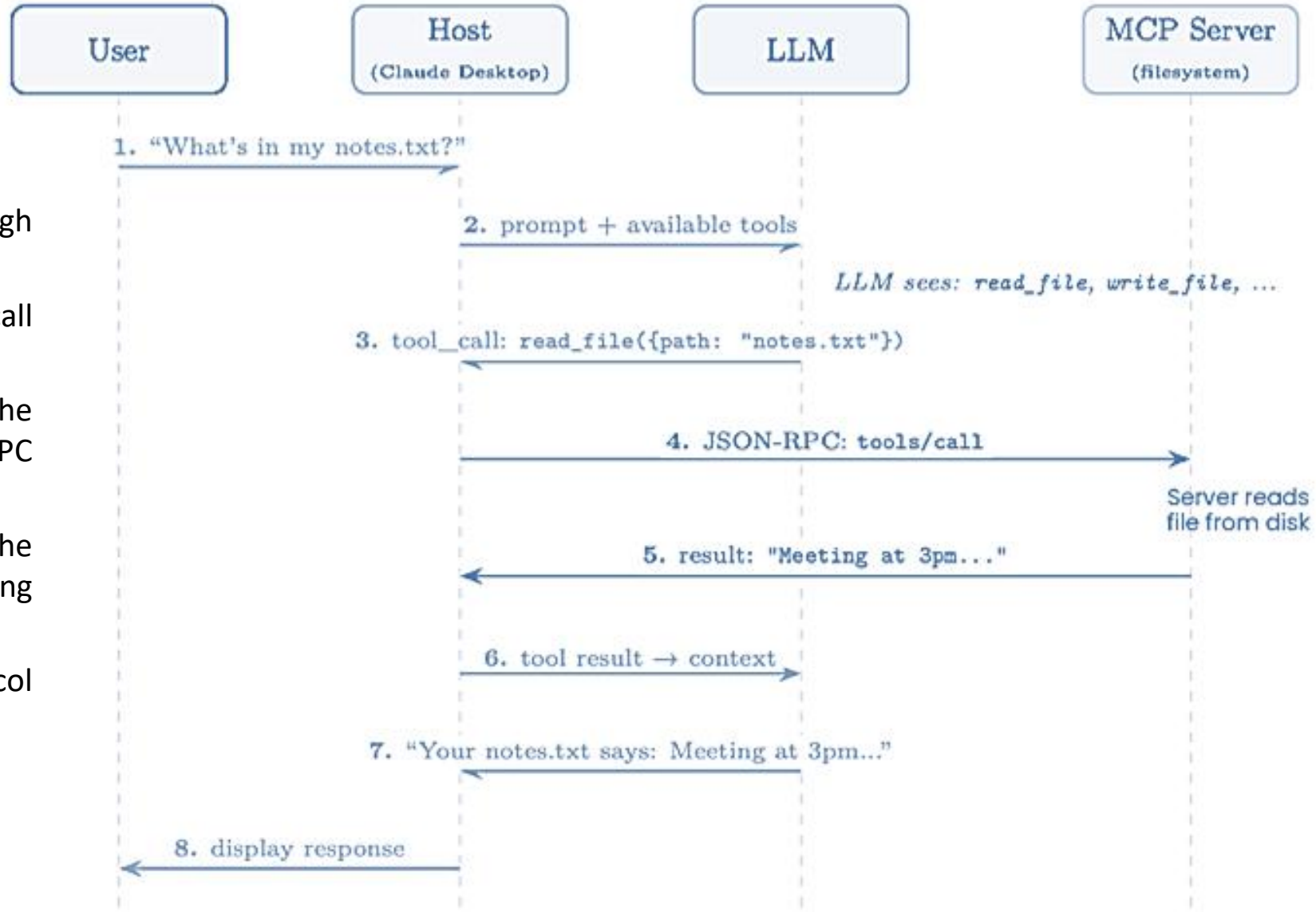


Fig: MCP architecture. The harness acts as an MCP client, routing tool calls to specialized MCP servers over standardized transports.[6]

How MCP works:

Fig: A single user request flows through the **Host**, **LLM**, and **MCP Server**.

- The LLM decides which tool to call (step 3);
- the Host routes the call to the appropriate server via JSON-RPC (step 4);
- the result flows back through the LLM for natural-language formatting (steps 5–7).
- The user never sees the protocol machinery. [6]

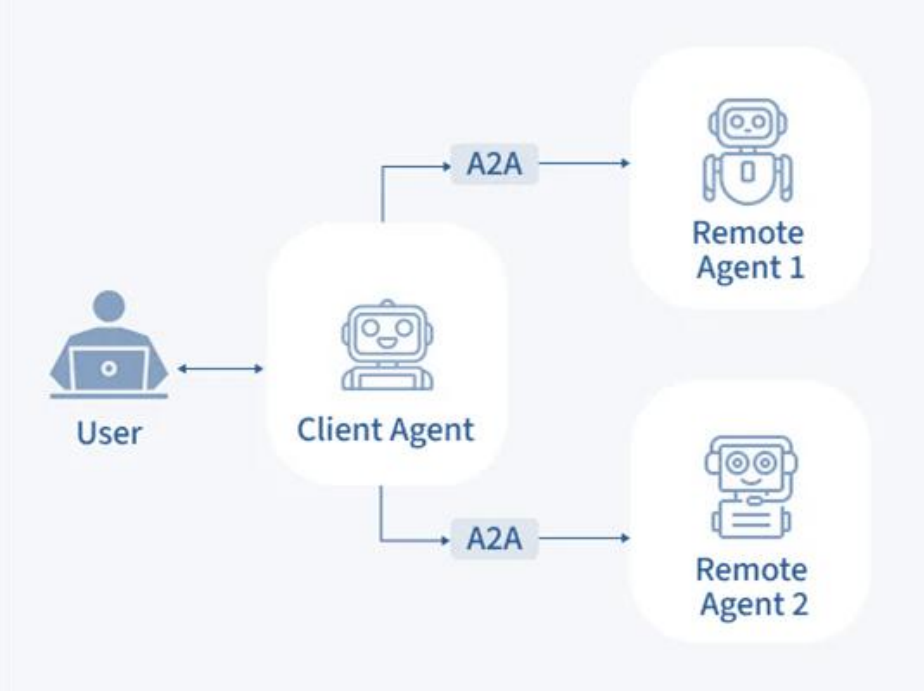


Agent-to-Agent Communication (A2A)

- **The Core Problem:** As large language models evolve from isolated assistants into collaborative networks of specialized agents, the question of how agents talk to each other becomes as important as how they reason internally.
- **The Solution (A2A):** In April 2025, Google (with contributions from over 50 technology partners) released the Agent to-Agent (A2A) Protocol [113], an open specification for interoperable communication between AI agents. [6]

A2A is built on five core principles:

- **Opaque Execution:** Calling agents interact exclusively through declared interfaces without inspecting remote internals. This abstracts underlying models (e.g., GPT-4, Gemini, rule-based systems) to support heterogeneous ecosystems.
- **Enterprise Readiness:** Protocol-level integration of OAuth 2.0, API keys, JWT, audit logging, and regulatory compliance from day one.
- **Modality Agnosticism:** Native support for combined text, binary files, and structured JSON in a single message—no protocol extensions required for multimodal inputs.
- **Leveraging Existing Standards:** Built on proven infrastructure using HTTP/HTTPS, JSON-RPC 2.0, Server-Sent Events (SSE) for streaming, and gRPC as an alternative transport binding.
- **Async-First Architecture:** Native support for long-running operations via push notifications and polling, eliminating open-connection overhead. [6]



Agent Skills: Composable Capabilities

What Is a Skill?

A skill is a self-contained capability module that gives an agent expertise in a specific domain or task. Unlike a raw tool (which exposes a single function), a skill encompasses:

- **System prompt augmentation:** Domain-specific instructions, constraints, and persona elements injected into the agent's context.
- **Tool bindings:** One or more tools the skill requires (APIs, MCP servers, local commands).
- **Knowledge:** Reference material, examples, or few-shot demonstrations the agent needs to execute the skill correctly.
- **Workflow logic:** Multi-step procedures, decision trees, or conditional flows that guide the agent through complex tasks.
- **Guardrails:** Skill-specific safety constraints, output format requirements, and validation rules. [6]

The Traditional SDLC

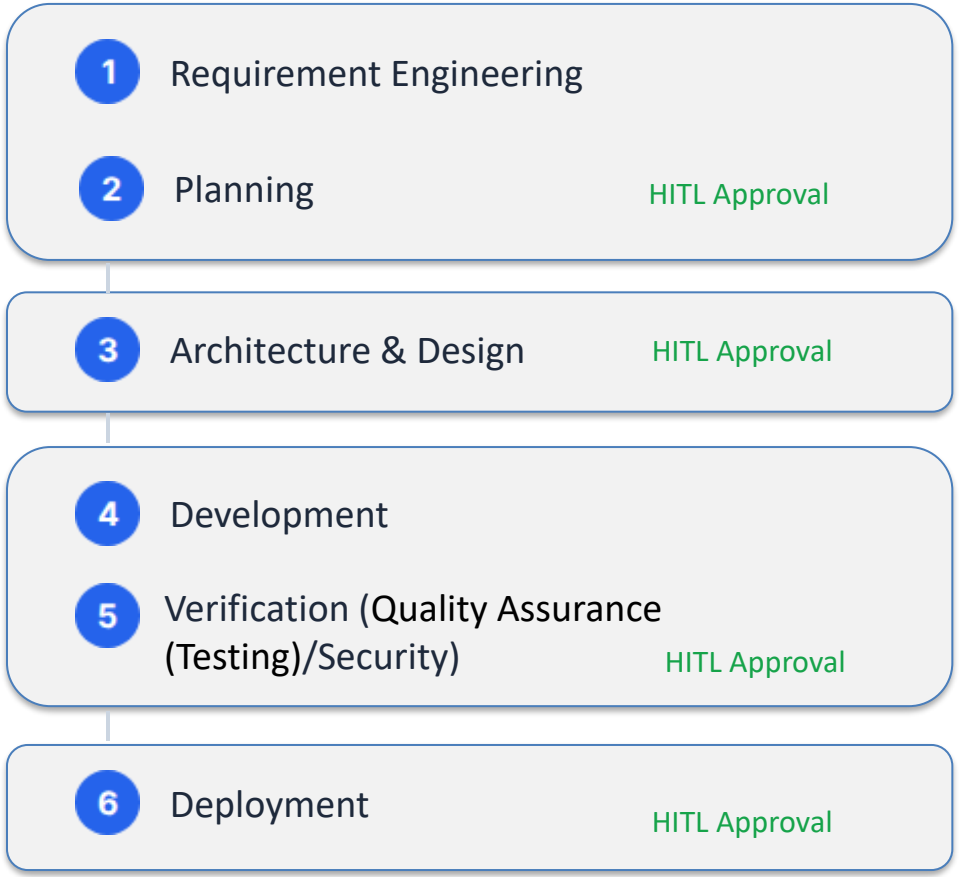
The Traditional Software Development Life Cycle (SDLC) is the structured framework that engineering teams follow to plan, design, build, test, and deploy software applications from initial concept.

This Manual SDLC execution creates severe friction—engineers waste most of their time on administrative handoffs, environment setups, regression testing, and ticketing overhead instead of writing core logic.[5]

So, Our Aim is to automate the Traditional SDLC with Agentic AI, which will replaces these static bottlenecks with continuous, self-healing pipelines that autonomously execute tests, patch build failures, flag vulnerabilities, and manage releases. This **eliminates human error, compresses cycle times from months to hours, and frees development teams to focus on high-value architecture and innovation.**

Proposed Architecture: Modular Pipeline

A modular pipeline with integrated **Human-in-the-Loop (HITL)** governance gates to ensure enterprise safety.

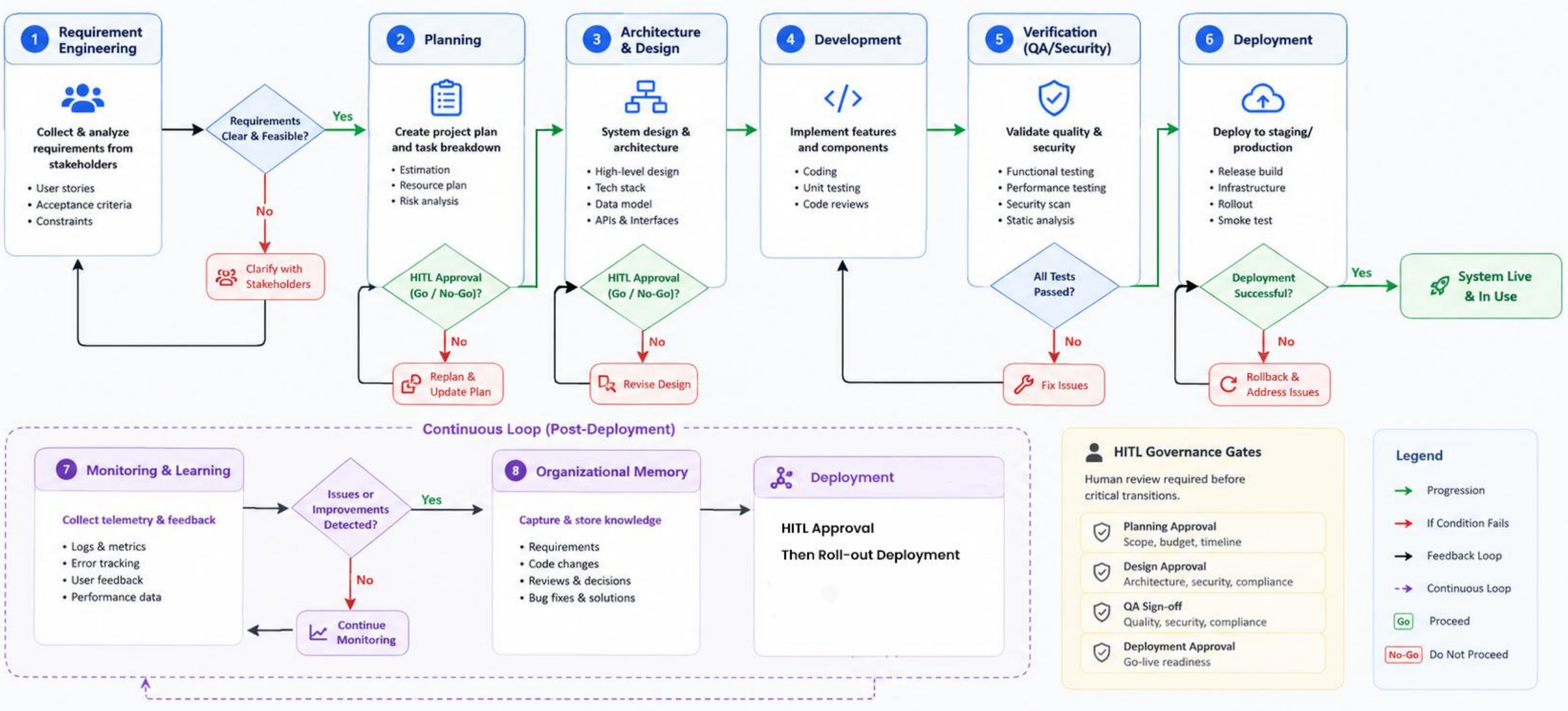


Continuous Loop

7. Monitoring & Learning: Consumes production logs, metrics, and user feedback to detect errors and optimize prompts.

8. Organizational Memory: Captures all requirements, code, reviews, and bug fixes into a Knowledge Graph for cross-sprint learning.

Proposed Architecture: Modular Pipeline



Core Innovation: Adaptive Team Generation

Traditional: Fixed Agents

Standard agentic systems rely on rigid, hardcoded workflows (e.g., always Planner → Developer → Tester).

- Inefficient for small tasks (over-engineered).
- Lacks specialized roles (Security, DevOps) for complex enterprise apps. [1]

Proposed: Dynamic Generation

The Orchestrator analyzes project complexity, risk, and size before execution to spawn agents on-the-fly.

- **Small Script:** Spawns Planner + Developer.
- **Enterprise App:** Spawns Architect, Backend, Frontend, Security, QA, DevOps.
- Agents are retired when tasks complete, saving compute.

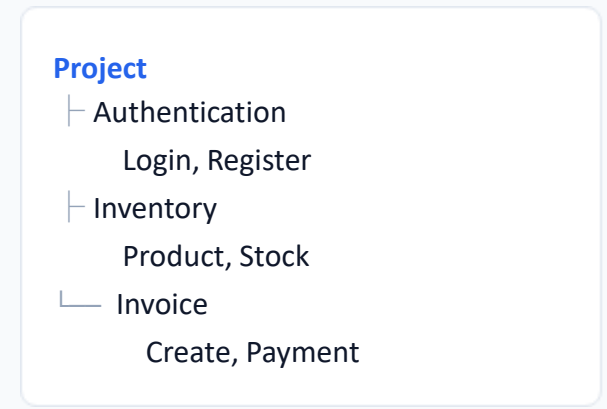
Core Innovation: Hierarchical Software Knowledge Graph (HSKG)

Beyond Traditional Memory:
A Context-Aware Project Memory

The Problem: Current AI agents retrieve entire repositories or large documents, resulting in high token consumption and irrelevant context. [1]

Our Solution: Our proposed HSKG stores the software project as a hierarchical graph, enabling agents to retrieve only the smallest relevant subgraph required for a task.

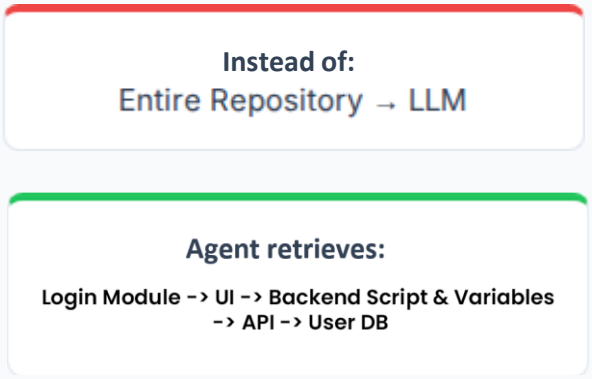
Project Graph



Purpose

- Represents the complete SDLC structure
- Maintains hierarchy between project components
- Enables modular navigation

Context-Aware



Benefits

- Minimal context
- Lower token usage
- Faster reasoning
- Better accuracy
- Reduced hallucination

Identifying the Gap

While highly capable, existing autonomous systems index heavily on code generation, lacking end-to-end SDLC orchestration and organizational memory.

System	Requirements	Design	Coding	Testing	Deployment
Devin AI [15]	Partial	Partial	Yes	Yes	Partial
MetaGPT / ChatDev [11][12]	Yes	Yes	Yes	Partial	No
SWE-Agent [13]	No	No	Yes	Yes	No
Proposed System	Yes	Yes	Yes	Yes	Yes

Source: *Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.*

Agentic Frameworks

Framework	Execution Model	Core Strengths	Key Limitations	Ideal Use Cases	Offline / Local Capability
LangChain [7]	Chained Prompting & Tools	Rich toolset, extensive RAG ecosystem, vast API integrations	Workflow complexity at scale, abstractions can hide errors	General LLM applications, retrieval-augmented generation	**Excellent:** Natively connects to local LLMs (Ollama, Llama.cpp) and local vector stores (Chroma).
LangGraph [8]	Stateful Graph Orchestration	Cycles and loops, robust state persistence, time-travel debugging	Steeper learning curve, excessive setup for simple tasks	Production-grade multi-agent systems, human-in-the-loop control	**Excellent:** Inherits LangChain's local capabilities; operates entirely offline with self-hosted models.
CrewAI [9]	Role-based Team Collaboration	Easy setup, intuitive agent personas, automatic task delegation	Limited dynamic branching and complex workflow flow control	Autonomous research pipelines, content creation, structured teams	**Strong:** Explicitly supports local execution by binding agents to local endpoints via Ollama or LiteLLM.
OpenAI Agents SDK [10]	Lightweight Handoffs & Guardrails	Minimal abstractions, built-in tracing, standardized routing primitives	Lacks built-in persistent long-term memory or native graph structures	Rapid deployment of production agents, voice pipelines, clear delegation	**Very Good:** Provider-agnostic design allows it to run offline by routing to local models via the Chat Completions API.

Source: *Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.*

Local Offline Execution Strategy

Deploying via offline models ensures data privacy. Model selection is based on VRAM footprint, Training data freshness, Suitable for Agentic Operations.

Model (4-bit Quant)	Recommended Hardware (VRAM)	Knowledge Cutoff	Agent Capability	Best Use Case
Gemma 4 4B	Laptop / iGPU (~5 GB)	Early 2026	⭐⭐⭐	Basic assistants, RAG, lightweight offline agents
Gemma 4 12B	RTX 4070 (8–10 GB)	Mid 2026	⭐⭐⭐⭐	General-purpose AI agents with good reasoning & tool use
Mistral Small 3.2 (24B)	RTX 4090 (16 GB)	Mid 2025	⭐⭐⭐⭐⭐	Enterprise agents, multi-tool workflows, planning
Qwen3-Coder 32B	Workstation / Mac M-Series (20–22 GB)	Mid 2025	⭐⭐⭐⭐⭐	Coding agents, debugging, software engineering
Llama 3.3 70B	Multi-GPU / High-End Server (48+ GB)	Late 2023*	⭐⭐⭐⭐⭐	Research, complex planning, multi-agent orchestration

Source: *Compiled from official model documentation and validated using publicly available implementation benchmarks and deployment experiences.*

Conclusion

We are proposing an **Autonomous Software Development Lifecycle (A-SDLC)** framework that transforms traditional software engineering into an **AI-Orchestrated Software Engineering Organization**.

Instead of isolated coding assistants, the system coordinates multiple specialized AI agents across the complete SDLC—from **requirements engineering** to **continuous monitoring and learning**.

Expected Outcomes

- Reduced context size and token consumption
- Faster and more accurate software development
- Improved traceability across the SDLC
- Persistent engineering knowledge

References

Research Papers & Book

[1] He, J., Treude, C., & Lo, D. (2025). *LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision...* ACM TOSEM.

[2] Apostolou, S. A., et al. (2026). *Assistance to Autonomy: A Systematic Literature Review of Agentic AI across the SDLC*. arXiv.

[3] *Agentic AI in the Software Development Lifecycle: Architecture, Empirical Evidence...* (2026). arXiv.

[4] *Veselka Sasheva Petrova-Dimitrova, "Classifications of intelligence agents and their applications," Fundamental sciences and applications, Vol. 28, No. 1, 2022.*

[5] Sommerville, I. *Software Engineering (10th Edition)*.

[6] The Hitchhiker's Guide to Agentic AI: From Foundations to Systems by Haggai Roitman

Articles & Official Documentations

[7] *LangChain Documentation*.

[8] *LangGraph Documentation*.

[9] *CrewAI Documentation*.

[10] *OpenAI Agents SDK Documentation*.

[11] Hong, S., et al. (2024). *MetaGPT: Meta Programming for Multi-Agent Collaborative Framework*.

[12] Qian, C., et al. (2024). *ChatDev: Communicative Agents for Software Development*.

[13] Yang, J., et al. (2024). *SWE-Agent: Agent-Computer Interfaces Enable Automated SE*.

[14] *OpenHands Project*.

[15] Cognition AI – *Devin: AI Software Engineer*.

[16] *Microsoft-copilot/for-individuals/do-more-with-ai/general-ai/understanding-ai-agents-vs-chatbots?*

[17] *IBM: https://www.ibm.com/think/topics/ai-agents*

[18] *Google Cloud*: *cloud.google.com/discover/what-is-model-context-protocol*

Thank You